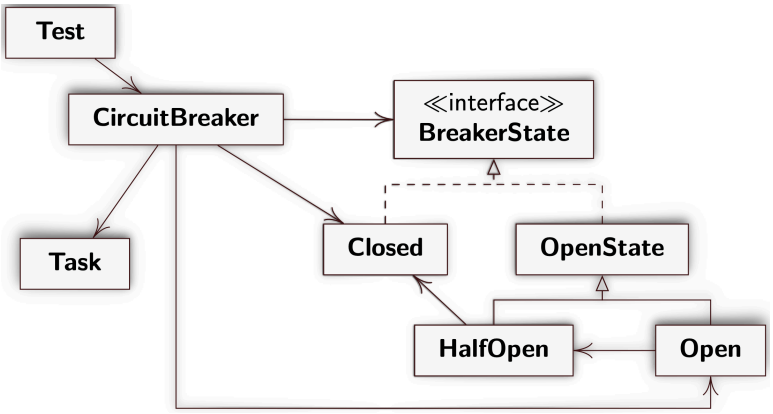
Esempio di Circuit Breaker

Case study Circuit Breaker

Il caso di studio illustra un servizio (rappresentato dalla classe Task) che durante l'esecuzione può subire errori o rallentamenti. Il pattern Circuit Breaker viene utilizzato per monitorare le richieste e le risposte del servizio Task, passando dinamicamente dallo stato di chiuso, ad aperto ed infine a semi-aperto, in base agli errori o ai rallentamenti rilevati a runtime. A livello architetturale, questo meccanismo è implementato sfruttando la struttura del design pattern **State**: la classe CircuitBreaker svolge il ruolo di *Context*, mentre le classi specifiche degli stati (Closed, HalfOpen e Open) ricoprono il ruolo di *ConcreteState*.



Dettagli del servizio

L'esecuzione del servizio è simulata all'interno del metodo nextStep() della classe Task. Questo metodo è progettato per simulare tre possibili scenari di comportamento:

→ restituire correttamente un valore in un tempo inferiore a 300 ms.

→ lanciare un'eccezione.

→ durare più del tempo massimo atteso (fissato a 300 ms).

A livello statistico, l'implementazione dell'esempio fa sì che le esecuzioni si dividano in terzi: un terzo lancia un'eccezione, un terzo subisce un ritardo randomico compreso tra i 200 e i 700 ms, e l'ultimo terzo ritorna il risultato immediatamente.

```
public class Task {
    private int step = 0;
    private static final String[] steps = {"stpZero","stpOne","stpTwo","st
pThree","stpFour"};

    private int nextStepOk() {
         System.out.print("In next step ... ");
         System.out.print("From " + steps[step] + " to ");
         step++;
         if (5 == step) step = 0;
         System.out.println(steps[step]+" ...");
         return step;
```

```
    }

    private static void randomDelay() {
         int delay = 200 + new Random().nextInt(500);
         try {
             Thread.sleep(delay);
         } catch (InterruptedException e) {
                 throw new RuntimeException(e);
         }
    }

    /**
     * Statistically, one third of the times the next step cannot be reach
ed, an Exception
     * occurs. When there is no Exception, there is a variable delay.
     * @return a value between 0 and 4 or throws Exception
     * @throws Exception
     **/
    public int nextStep() throws Exception {
         System.out.print("Try to go to next step ... ");
         int randomVal = new Random().nextInt(3);
         if (0 == randomVal) throw new Exception();
         else if (1 == randomVal) randomDelay();
         return nextStepOk();
    }
}
```

Dettagli di Circuit Breaker

Le richieste rivolte al servizio non vanno direttamente a Task, ma arrivano alla classe CircuitBreaker invocando il metodo getResultBoundTime(). Il suo funzionamento si articola in questo modo:

- **Circuito chiuso**: se il circuito è nello stato chiuso, il servizio viene avviato in modo asincrono utilizzando supplyAsync().
- **Apertura del circuito** (fallimento): se il risultato della chiamata non si rende disponibile entro un timeout di 300 ms (usando il metodo get() con timeout), o se la chiamata lancia un'eccezione, il Circuit Breaker deduce che il servizio non sta eseguendo bene e transita nello stato aperto.
- **Transizione semi-aperto**: una volta aperto, il circuito resta in tale stato per 200 ms. Scaduto questo tempo, passa allo stato semi-aperto e vi rimane per 100 ms.
- **Comportamento semi-aperto**: nello stato semi-aperto, l'interruttore lascia passare solo la metà delle richieste. Ogni volta che giunge una nuova richiesta, il sistema valuta se è trascorso il tempo necessario per passare allo stato successivo (da aperto a semi-aperto, o da semi-aperto per tornare a chiuso).

```
public interface BreakerState { // role State
    public BreakerState nextState();
    public boolean isClosed();
}
```

```
public class Closed implements BreakerState { // role ConcreteState
    public Closed() {
        System.out.println("go to closed");
    }

    @Override
    public BreakerState nextState() {
        return this;
    }

    @Override
    public boolean isClosed() {
        return true;
    }
}
```

```
public abstract class OpenState implements BreakerState {
    private long openTime;
    private long deltaTimeOpen;
    private String st;

    public OpenState(long delta, String name) {
        openTime = System.currentTimeMillis();
        deltaTimeOpen = delta;
        st = name;
        System.out.println("go to " + st);
    }

    /**
     * if enough time has passed, go to next state
     * @param s a Supplier giving the next state
     */
    public BreakerState nextState(Supplier<BreakerState> s) {
        long elapsed = System.currentTimeMillis() - openTime;
        System.out.print("time in " + st + " " + elapsed + " ms, ");
        if (elapsed < deltaTimeOpen) {
            System.out.println("stay " + st);
            return this;
    }
    return s.get(); // change CircuitBreaker’s state
    }
}
```

```
public class HalfOpen extends OpenState { // role ConcreteState
    public HalfOpen() {
        super(100, "half open"); // half open for 0.1 sec
```

```
    }

    @Override
    public BreakerState nextState() {
        return nextState(() -> new Closed()); // supplies next state
    }
    /**
     * returns true half of the times, statistically
     */
    @Override
    public boolean isClosed() {
        return (0 == new Random().nextInt(2));
    }
}

public class Open extends OpenState { // role ConcreteState
    public Open() {
        super(200, "open"); // open for 0.2 sec
    }

    @Override
    public BreakerState nextState() {
        return nextState(() -> new HalfOpen()); // supplies next state
    }

    @Override
    public boolean isClosed() {
        return false;
    }
}
```

```
public class CircuitBreaker { // role Context
    private Task t;
    private BreakerState state = new Closed(); // initial CircuitBreaker’s
state
    private CompletableFuture<Integer> task;

    public CircuitBreaker() {
        t = new Task();
    }

    public int getResultBoundTime() {
        state = state.nextState(); // check for change of CircuitBreaker’s
state
        if (!state.isClosed()) return -1; // request to service not perfor
med
        System.out.println("CircuitBreaker executing task");
```

```
        task = CompletableFuture.supplyAsync(() -> { // start async reques
t
            try {
                return t.nextStep(); // request a service
            } catch (Exception e) { // capture exception thrown by service
                System.out.println("An exception occurred in Task");
                state = new Open(); // change CircuitBreaker’s state
                return -1;
            }
        });
        return getValueFromFuture();
    }

    private int getValueFromFuture() {
        int result = -1;
        try {
            result = task.get(300, TimeUnit.MILLISECONDS); // wait up to 3
00 ms
        } catch (InterruptedException e) {
        } catch (ExecutionException e) {
        } catch (TimeoutException e) { // if result is not ready
            System.out.println("A timeout occurred in getting the resul
t");
            state = new Open(); // change CircuitBreaker’s state
        }
        return result;
    }
}
```

Dettagli del test

Per verificare il corretto funzionamento, il test effettua 30 chiamate al servizio passando sempre per il CircuitBreaker.

→ viene inserito un ritardo massimo di 0,1 secondi tra una chiamata e la successiva.

→ ad ogni chiamata, il CircuitBreaker restituisce un valore numerico tra 0 e 5 se l’esecuzione del servizio è andata a buon fine, altrimenti restituisce -1 per indicare un fallimento.

→ il test conta infine quanti sono i valori restituiti diversi da -1, in modo da determinare esattamente quante volte il servizio è riuscito ad eseguire la sua operazione in modo normale.

```
public class TestBreaker {
    public static void main(String[] args) {
        System.out.println("Calling method ");
        long c = callMethodGet();
        System.out.println("Number of successful calls " + c);
    }

    private static long callMethodGet() {
```

```
        CircuitBreaker brk = new CircuitBreaker();
        // call with 0.1 sec max intervals for 30 times
        long c = IntStream.rangeClosed(1, 30).map(i -> {
            delay01sec();
            return brk.getResultBoundTime(); // request service
        }).filter(v -> v > -1).count(); // count actual results
        return c;
    }

    private static void delay01sec() {
        try {
            Thread.sleep(new Random().nextInt(100));
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
}
```